DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

QUICK REFERENCE

ODT-REF

USAREUR-AF

DoctrineElement → Palantir Foundry Mapping

1. Object Type Identity

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

16 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DOCTRINEELEMENT → PALANTIR FOUNDRY MAPPING

Authoritative mapping between the GDAP `DoctrineElement` Python schema and the Palantir Foundry Object Type representation, including TypeScript property names, OSDK types, and bi-temporal semantics.

Last updated: 2026-03-12

1. OBJECT TYPE IDENTITY

Attribute	Value
Foundry API name	`DoctrineElement`
Foundry namespace	`gdap`
OSDK interface	`Osdk.Instance<DoctrineElement>`
Primary key property	`id` (string)
Primary key source	`DoctrineElement.unique_id`

2. PRIMARY KEY PATTERN

The `unique_id` field encodes five hierarchical coordinates:

```
{nation}:{document}:{section}:{paragraph}:{version}
```

Example: `US:ADP 3-0:Chapter 2:2-14:v2.0`

This value maps directly to the OSDK `id` property:

```
const element = await client.objects.DoctrineElement.get("US:ADP 3-0:Chapter 2:2-14:v2.0");
```

Rule: The separator is `:`. Each segment must be non-empty. The validator enforces at least 5 segments (split on `:`).

3. PROPERTY MAPPING TABLE

Core Identity

Python Field	Foundry Property	TypeScript Type	Notes
`unique_id`	`id`	`string`	Primary key; 5-segment colon-separated path
`source_nation`	`sourceNation`	`string`	`SourceNation` enum value (e.g., `"US"`, `"NATO"`)
`source_document`	`sourceDocument`	`string`	Document identifier (e.g., `"ADP 3-0"`)
`source_section`	`sourceSection`	`string`	Section reference string
`version`	`version`	`string`	Document version string

Content

Python Field	Foundry Property	TypeScript Type	Notes
`content_type`	`contentType`	`string`	`ContentType` enum value (e.g., `"definition"`)
`content_text`	`contentText`	`string`	Minimum 1 character; the doctrine text atom
`related_concepts`	`relatedConcepts`	`string[]`	Array of concept strings
`related_tasks`	`relatedTasks`	`string[]`	Array of task strings
`data_implications`	`dataImplications`	`string[]`	Array of data implication strings
`warfighting_function`	`warfightingFunction`	`string | undefined`	`WarfightingFunction` enum value

DRAFT

Structural Provenance

Python Field	Foundry Property	TypeScript Type	Notes
`parent_document_id`	`parentDocumentId`	`string`	References `DoctrineDocument.document_id`
`parent_section_id`	`parentSectionId`	`string | undefined`	References `DoctrineSection.section_id`

Extraction Metadata

Python Field	Foundry Property	TypeScript Type	Notes
`confidence_score`	`confidenceScore`	`double`	Range [0.0, 1.0]
`extraction_method`	`extractionMethod`	`string`	`ExtractionMethod` enum (e.g., `"llm_assisted"`)

DRAFT

DVEE Extensions

Python Field	Foundry Property	TypeScript Type	Notes
`authority_level`	`authorityLevel`	`string | undefined`	Command authority tier
`escalation_required`	`escalationRequired`	`boolean | undefined`	Whether escalation is mandated
`escalation_tier`	`escalationTier`	`string | undefined`	Tier label for escalation
`time_sensitivity`	`timeSensitivity`	`string | undefined`	Decision latency category
`risk_acceptance_level`	`riskAcceptanceLevel`	`string | undefined`	Risk tolerance label
`legal_trigger`	`legalTrigger`	`boolean | undefined`	Whether element triggers legal review
`cross_domain_dependency`	`crossDomainDependency`	`string | undefined`	Cross-domain dependency identifier
`decision_latency_estimate`	`decisionLatencyEstimate`	`string | undefined`	Human-readable latency estimate

4. BI-TEMPORAL FIELDS

GDAP uses a bi-temporal model with two independent time axes:

Transaction Time (when the record was created)

Python Field	Foundry Property	TypeScript Type	Notes
`extracted_at`	`extractedAtTimestamp`	`timestamp`	ISO 8601 UTC; auto-populated at extraction time

Valid Time (when the doctrine is in effect)

Python Field	Foundry Property	TypeScript Type	Notes
`effective_date`	`effectiveDate`	`date`	ISO 8601 date; from the source document
`valid_from`	`validFrom`	`date | undefined`	Start of doctrinal validity window
`valid_to`	`validTo`	`date | undefined`	End of doctrinal validity window (`null` = still valid)
`superseded_at`	`supersededAt`	`timestamp | undefined`	When this element was replaced by a newer version

Bi-temporal semantics:

- A record is **currently valid** when `valid_from <= today <= valid_to` (or `valid_to` is null) and `superseded_at` is null.
- A record is **historically accurate** as of transaction time `T` when `extracted_at <= T`.
- To reconstruct doctrine as-of a past date `D`, filter: `effective_date <= D` and `superseded_at > D` (or null).

5. NAMING CONVENTION

All Python `snake_case` field names are converted to Foundry `camelCase` property API names:

```
snake_case        → camelCase
effective_date    → effectiveDate
extracted_at      → extractedAtTimestamp  (suffix added to clarify it is a timestamp)
```

Object type API names use `PascalCase`:

```
DoctrineElement  →  DoctrineElement
DoctrineDocument →  DoctrineDocument
DoctrineSection  →  DoctrineSection
```

6. FOUNDRY TYPESCRIPT INTERFACE

```
import { defineObjectType } from "@osdk/maker";

export const DoctrineElement = defineObjectType({
  apiName: "DoctrineElement",
  displayName: "Doctrine Element",
  description: "Atomic unit of doctrine extracted from a publication.",
  primaryKey: "id",
  properties: {
    id: { type: "string", displayName: "Unique ID" },
    sourceNation: { type: "string", displayName: "Source Nation" },
    sourceDocument: { type: "string", displayName: "Source Document" },
    sourceSection: { type: "string", displayName: "Source Section" },
    version: { type: "string", displayName: "Version" },
    contentType: { type: "string", displayName: "Content Type" },
    contentText: { type: "string", displayName: "Content Text" },
    relatedConcepts: { type: "string", array: true, displayName: "Related Concepts" },
    relatedTasks: { type: "string", array: true, displayName: "Related Tasks" },
    dataImplications: { type: "string", array: true, displayName: "Data Implications"
},
    warfightingFunction: { type: "string", displayName: "Warfighting Function" },
    parentDocumentId: { type: "string", displayName: "Parent Document ID" },
    parentSectionId: { type: "string", displayName: "Parent Section ID" },
    confidenceScore: { type: "double", displayName: "Confidence Score" },
    extractionMethod: { type: "string", displayName: "Extraction Method" },
    effectiveDate: { type: "date", displayName: "Effective Date" },
    extractedAtTimestamp: { type: "timestamp", displayName: "Extracted At" },
    validFrom: { type: "date", displayName: "Valid From" },
    validTo: { type: "date", displayName: "Valid To" },
    supersededAt: { type: "timestamp", displayName: "Superseded At" },
    authorityLevel: { type: "string", displayName: "Authority Level" },
    escalationRequired: { type: "boolean", displayName: "Escalation Required" },
    escalationTier: { type: "string", displayName: "Escalation Tier" },
    timeSensitivity: { type: "string", displayName: "Time Sensitivity" },
    riskAcceptanceLevel: { type: "string", displayName: "Risk Acceptance Level" },
    legalTrigger: { type: "boolean", displayName: "Legal Trigger" },
    crossDomainDependency: { type: "string", displayName: "Cross Domain Dependency" },
    decisionLatencyEstimate: { type: "string", displayName: "Decision Latency
Estimate" },
  },
});
```

7. OSDK QUERY PATTERNS

Get a single element by ID

```
const elem = await client.objects.DoctrineElement.get(
  "US:ADP 3-0:Chapter 2:2-14:v2.0"
);
```

Query by nation and document

```
const usElements = await client.objects.DoctrineElement
  .where({ sourceNation: { eq: "US" }, sourceDocument: { eq: "ADP 3-0" } })
  .fetchPage({ pageSize: 100 });
```

Query currently valid elements (bi-temporal)

```
const today = new Date().toISOString().slice(0, 10);

const validNow = await client.objects.DoctrineElement
  .where({
    effectiveDate: { lte: today },
    supersededAt: { isNull: true },
  })
  .fetchPage({ pageSize: 200 });
```

Query by warfighting function

```
const firesElements = await client.objects.DoctrineElement
  .where({ warfightingFunction: { eq: "fires" } })
  .fetchPage({ pageSize: 50 });
```

DRAFT

8. LINK TYPES

Link Name	Source	Target	Cardinality	Notes
`containedBy`	`DoctrineElement`	`DoctrineDocument`	many-to-one	Via `parentDocumentId`
`inSection`	`DoctrineElement`	`DoctrineSection`	many-to-one	Via `parentSectionId`
`alignedTo`	`DoctrineElement`	`DoctrineElement`	many-to-many	Via `AlignmentPair` join

9. ENUM REFERENCE

SourceNation

Python Value	Foundry String
`SourceNation.US`	`"US"`
`SourceNation.GB`	`"GB"`
`SourceNation.NATO`	`"NATO"`
`SourceNation.CA`	`"CA"`
`SourceNation.AU`	`"AU"`
`SourceNation.DE`	`"DE"`
`SourceNation.FR`	`"FR"`
`SourceNation.NL`	`"NL"`

DRAFT

ContentType

Python Value	Foundry String
`ContentType.definition`	`"definition"`
`ContentType.task`	`"task"`
`ContentType.authority`	`"authority"`
`ContentType.process`	`"process"`
`ContentType.information_requirement`	`"information_requirement"`
`ContentType.concept`	`"concept"`
`ContentType.constraint`	`"constraint"`

WarfightingFunction

Python Value	Foundry String
`WarfightingFunction.movement_maneuver`	`"movement_maneuver"`
`WarfightingFunction.fires`	`"fires"`
`WarfightingFunction.intelligence`	`"intelligence"`
`WarfightingFunction.sustainment`	`"sustainment"`
`WarfightingFunction.protection`	`"protection"`
`WarfightingFunction.command_control`	`"command_control"`
`WarfightingFunction.information`	`"information"`

DRAFT